Centro Universitário de Excelência
Sistemas de Informação

MecâniQA: da Ordem de Serviço ao Pedido de Peças

Autores: Hélio Nunes Moraes Junior
Kevilyn Bitencourt da Cruz
Marcelo Henrique Fernandes Mauricio
Murillo Rigaud dos Santos Oliveira
Rodrigo Alves Oliveira

MecâniQA
AUTOMOTIVE TECH

Vitória da Conquista, 2026

Agenda

MecâniQA
AUTOMOTIVE TECH

Objetivo: mostrar como a MecâniQA passou a controlar Ordens de Serviço e Pedidos de Peças com uma API segura, testada e pronta para crescer.

1. Contexto e objetivo:
O problema da MecâniQA e a evolução do MVP

2. Fundamentos de OO:
Construtores, this e relacionamentos

3. Entidade associativa:
Pedido com várias peças e multiplicidades

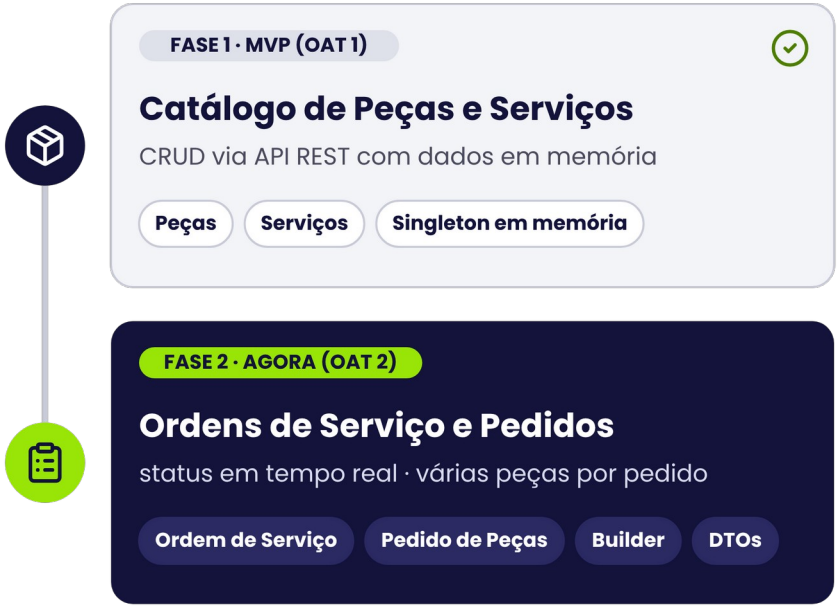
4. Padrões de projeto:
Builder, DTO e Mapper na prática

5. Fluxo e resultados:
Diagrama de atividade da US04 e entregas

O problema da MecâniQA

- **Problema:** a base de clientes cresceu e a gestão de estoque e serviços precisava migrar para a Web.
- **MVP da Fase 1:** API do catálogo de Peças e Serviços, com os dados em memória.
- **Fase 2 (esta entrega):** Ordens de Serviço e Pedidos de Peças, com status do orçamento à entrega.

Ganho para a oficina: atendente, estoquista e gerente acompanham o mesmo status em tempo real.



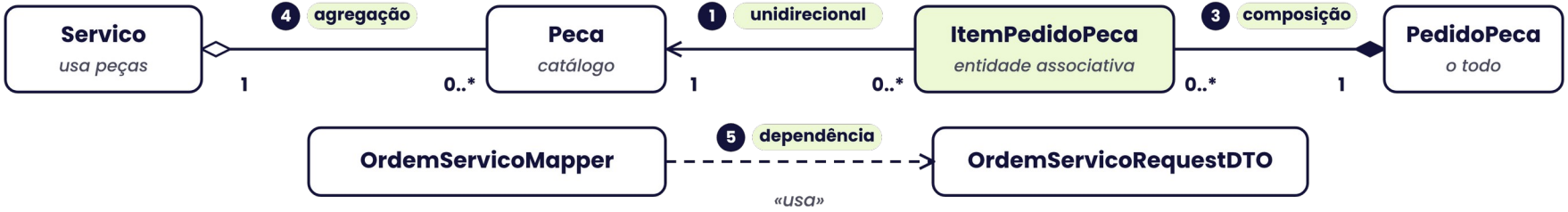
Construtores e this

- **Múltiplos construtores:** a mesma classe oferece mais de uma forma de criar o objeto.
- **Vazio:** o objeto nasce em branco e é preenchido pelos setters, como os Mappers fazem com Peca.
- **Completo:** ItemPedidoPeca(peca, quantidade) já nasce pronto para o pedido.
- **this:** aponta para o próprio objeto e separa o atributo do parâmetro de mesmo nome.

Ganho para a oficina: o preço do item fica registrado no pedido, mesmo se a peça for reajustada.



Relacionamentos



1 Unidirecional

Só um lado conhece o outro.

Item → Peca: a peça não conhece os itens.

2 Bidirecional

Os dois lados se conhecem (sem setas).

Evitado no projeto: geraria ciclo no JSON.

3 Composição

Todo-parte forte: a parte morre com o todo.

Pedido ◆ Item: o item só existe no pedido.

4 Agregação

Todo-parte fraco: a parte vive sozinha.

Serviço ◇ Peça: a peça segue no catálogo.

5 Dependência

Uso temporário, sem guardar como atributo.

Mapper ⇢ DTO: usado só dentro do método.

Entidade associativa



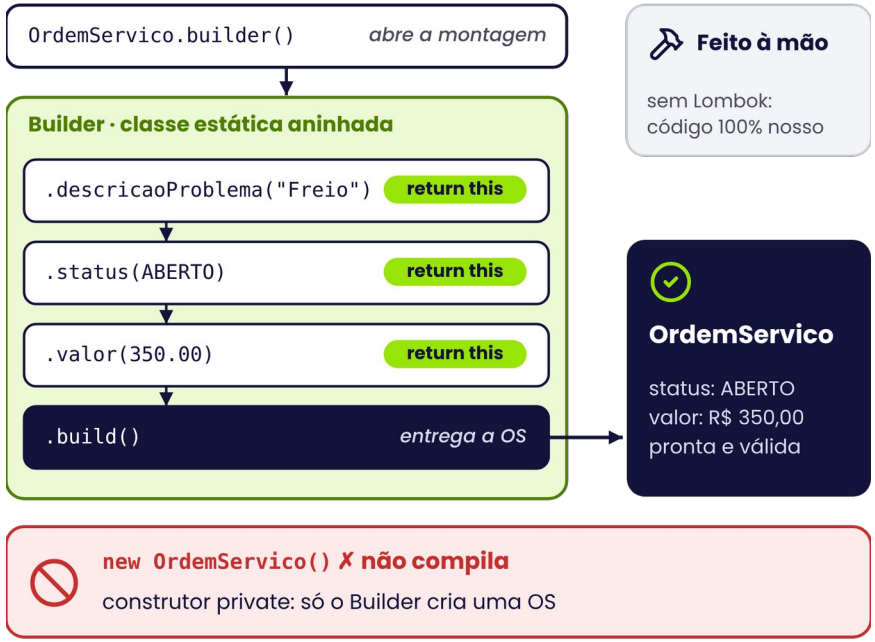
- **Muitos-para-muitos:** um pedido tem várias peças e uma peça aparece em vários pedidos.
- **ItemPedidoPeca** fica no meio e guarda o que é da relação: quantidade e preço unitário.
- **Multiplicidades:** um pedido tem 0..* itens e cada item aponta para 1 peça.
- **Atributos viram mapeamento:** itens (List) = 0..* e peca (Peca) = 1.

Ganho para a oficina: o total do pedido é calculado sozinho, somando os subtotais.

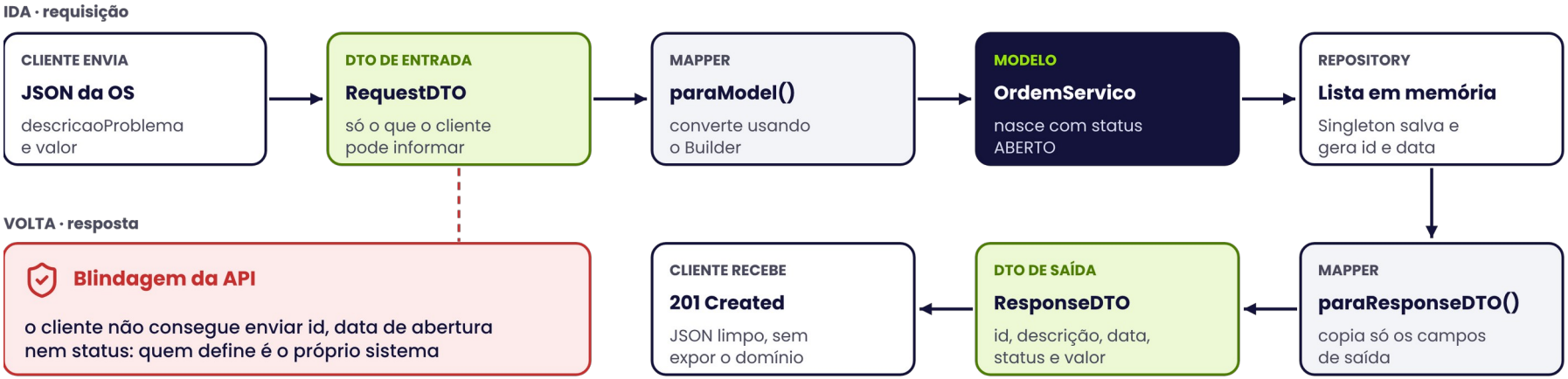
Padrão Builder

- **O que é:** padrão criacional que monta um objeto complexo passo a passo.
- **Por que usamos:** evita um construtor gigante e toda OS nasce consistente, com status ABERTO.
- **Como fizemos:** classe estática aninhada Builder, métodos com return this e build().
- **Blindagem:** construtor private; new OrdemServico() nem compila.

Ganho para a oficina: novos campos na OS entram com um método a mais, sem quebrar o que já funciona.



DTO e Mapper



DTO

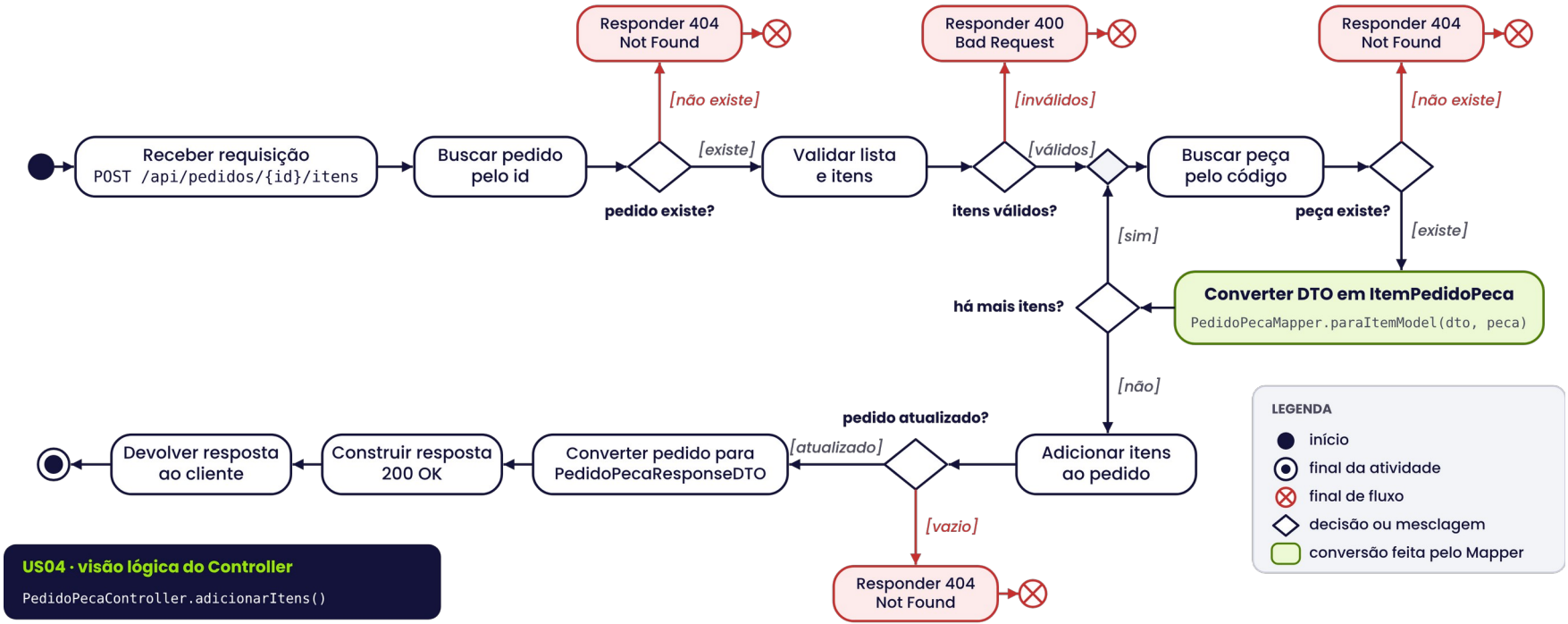
Classe só de transporte: leva e traz apenas os campos que o cliente precisa.

Mapper

Classe utilitária, com métodos static e sem estado, que converte DTO em modelo e vice-versa.

Ganho: contrato estável e seguro para quem consome a API.

Diagrama de Atividade



Considerações finais

5
User Stories entregues (US01 a US05)

84
testes automatizados sem nenhuma falha

- **Entregamos:** OS e Pedidos com status, entidade associativa, Builder, DTOs e Mappers.
- **Para a oficina:** menos retrabalho, status visível para toda a equipe e total automático.
- **Qualidade:** cada User Story tem cenários de sucesso e de erro testados.

41
chamadas no Postman com 126 verificações

0
uso de Lombok: tudo feito à mão

MecâniQA AUTOMOTIVE TECH

Referências

- MECÂNIQA. OAT 2 – Padrão de Projeto e Diagrama de Atividade. 2026.
- EQUIPE PORTO ALEGRE. Repositório do projeto, 2026. github.com/Meizolas/mecaniQA-api-porto-alegre
- GAMMA, E. et al. Padrões de Projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.
- FOWLER, M. Padrões de Arquitetura de Aplicações Corporativas. Porto Alegre: Bookman, 2006.
- OMG. Unified Modeling Language (UML) Specification, versão 2.5.1. 2017.
- SPRING. Spring Boot Reference Documentation. docs.spring.io/spring-boot